Final Project Report

Name: Kadiatou Diallo

Prof: Yanilda

Date: 5/6/2026

Class: CIS344

Title: Real Estate Agency Portal

1. Introduction

The purpose of this project is to develop a web-based Real Estate Agency Portal using PHP and MySQL. The platform simulates a real-world real estate system where agents, buyers, and renters interact with property listings.

The system allows:

- Agents to add and manage property listings
- Buyers and renters to browse available properties
- Users submit inquiries and save favorite properties
- The system to track transactions such as property sales and rentals

This project demonstrates the integration of frontend PHP pages with a backend MySQL database, along with authentication, session management, and role-based access control.

2. Database Design

The database used in this project is named: **real_estate_portal_db**

It consists of five main tables:

Users Table: This table stores all registered users of the system.

userId: Primary key
userName: Unique username
passwordHash: Encrypted password
userType: Defines role (agent, buyer, renter)
The purpose of this table is to store all real estate listings

Inquiries Table

This table stores messages from buyers or renters about properties.

inquiryId: Primary key, userId: Foreign key, propertyId: Foreign key, and message, inquiryDate

The purpose of this table is to track user interest in properties

Transactions Table

This table records completed property sales or rentals.

transactionId: Primary key

transactionType: sale or rental

amount, transactionDate

The purpose of this table is to maintain financial and transaction records

Favorites Table

This table stores saved properties for buyers and renters.

- favoriteId: Primary key
- userId, propertyId

The purpose of this table allows users to bookmark properties

Relationships

- One **agent** can manage multiple properties
- One **user** can submit multiple inquiries
- One **property** can have multiple inquiries
- Transactions link users and properties

View: PropertyListingView

- This view combines property and agent information to display listings in a user-friendly format.

Stored Procedures

- **AddOrUpdateUser**: Inserts or updates user records
- **ProcessTransaction**: Inserts transaction and updates property status

Trigger

- **AfterTransactionInsert** automatically updates property status when a transaction occurs

3. PHP Functionality

The application uses PHP with PDO for database interaction.

addUser()

- Insert a new user into the Users table
- Uses prepared statements for security

addProperty ()

- Allows agents to add new property listings
- Stores property details in the database

addInquiry ()

- Allows buyers/renters to submit messages about a property

getUserDetails ()

- Retrieves user data from the database
- Can be extended to include related records

Other Functionalities

- Viewing properties using PropertyListingView
- Saving favorites
- Managing sessions and dashboards

4. Login and session handling

The system implements secure authentication using PHP built-in functions.

Password Hashing

Passwords are securely stored using password_hash ($password, PASSWORD_DEFAULT);

Password Verification

During login: password_verify ($password, $hashedPassword);

Session Management

- PHP sessions are used to track logged-in users
- Session variables: userId, role

This helps Secure login handling, Persistent user sessions, Protection against unauthorized access.

5. Role-Based Access Control

The system supports three roles:

Agent: Add and manage properties, View listings

Buyer: Browse properties, submit inquiries, Save favorites

Renter: Same permissions as buyers but focused on rentals

Access is restricted by using role checks such as: requireRole(['agent']);

This ensures users can only access features relevant to their role.

6. Challenges and Solution

1. Login Issues

Problem: Users could not log in due to incorrect password comparison

Solution: Used password_verify () instead of plain text comparison

2. Foreign Key Errors

Problem: Inserting records failed due to missing referenced IDs

Solution: Ensured users and properties exist before inserting dependent data

3. Session Handling

Problem: Sessions were not persisting across pages

Solution: Added session_start() at the top of every protected page

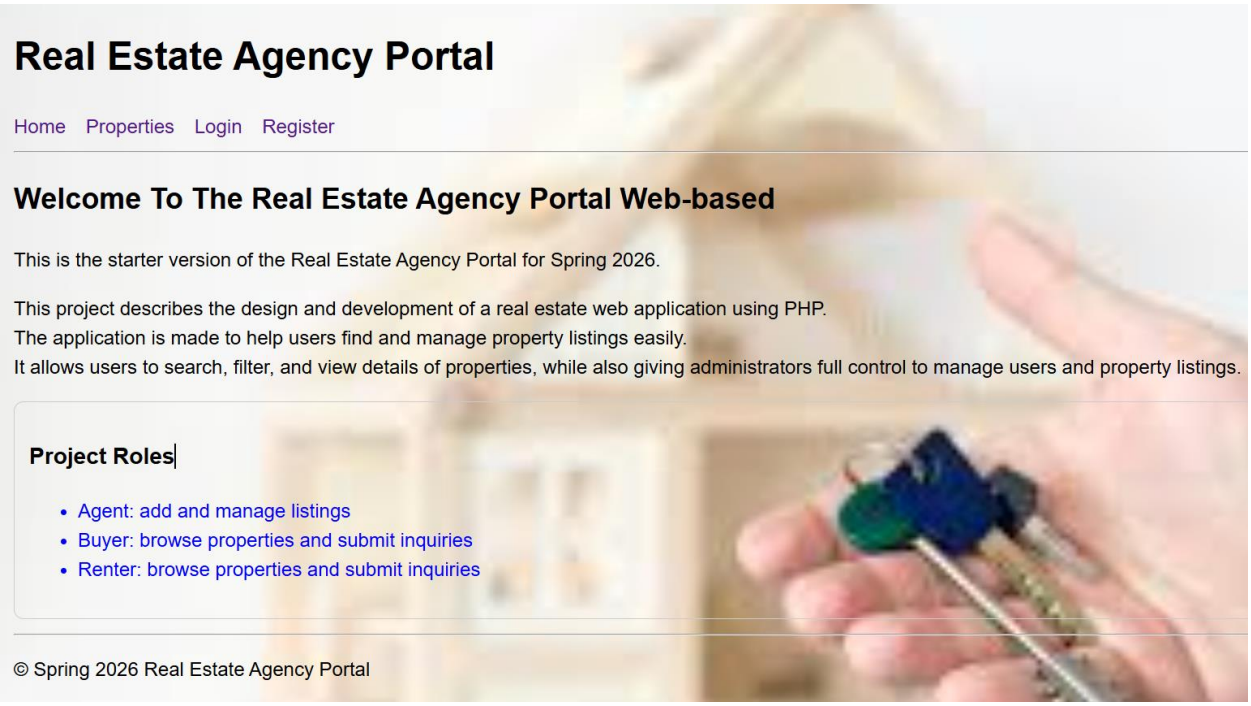
4. Role Restrictions

Problem: Users could access unauthorized pages

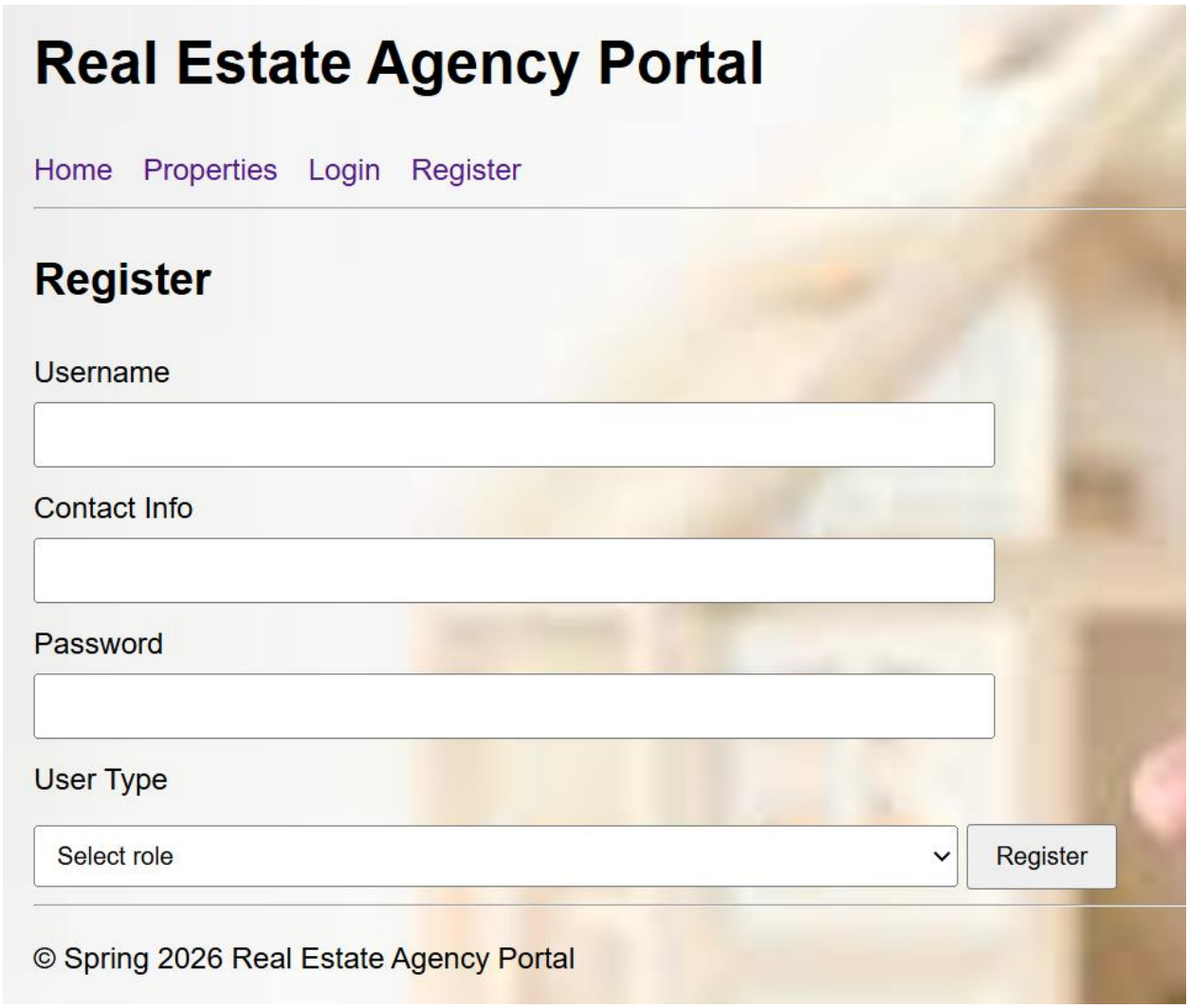
Solution: Implemented a requireRole () function

7. Screenshots

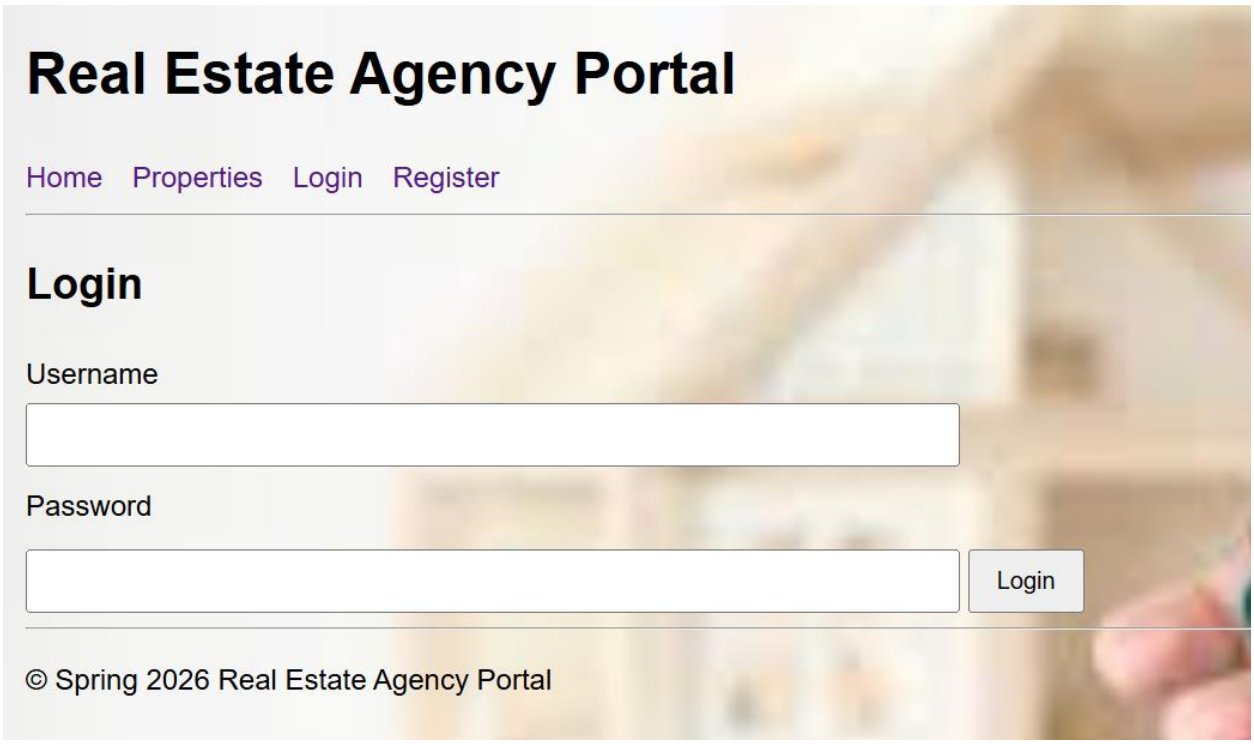
The following screenshots demonstrate the working system:

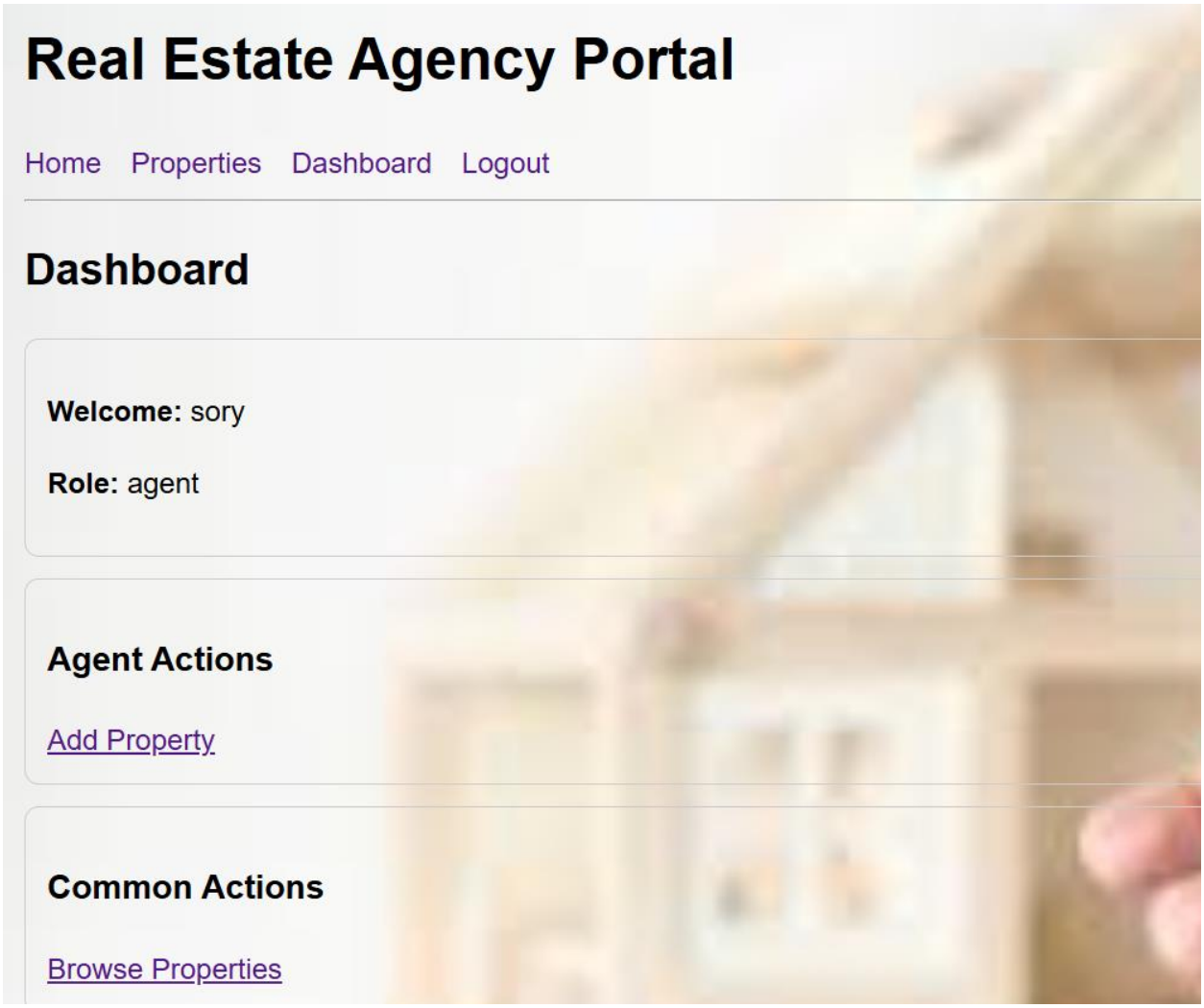


Register

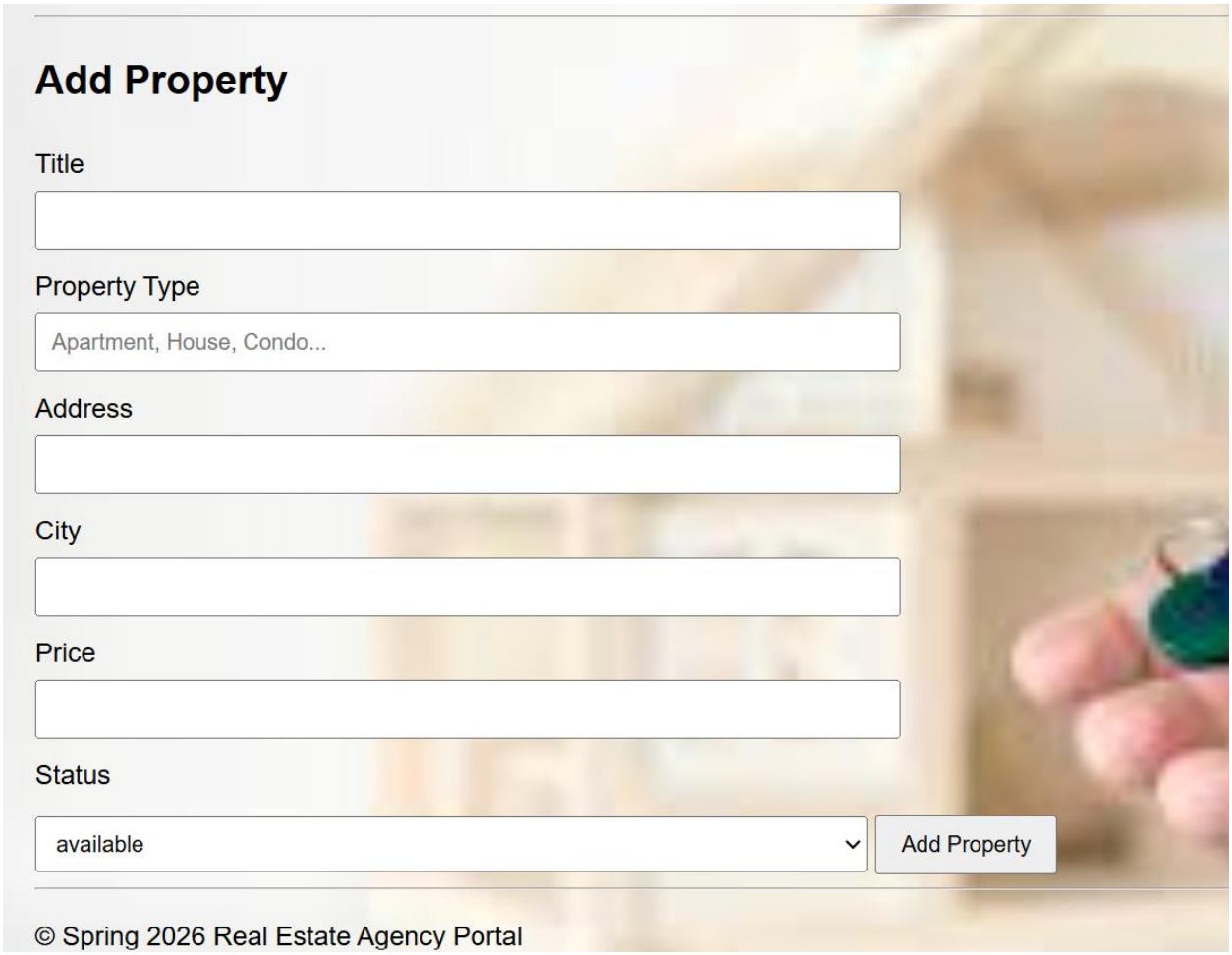

Real Estate Agency Portal
Home Properties Login Register
Register
Username
Contact Info
Password
User Type
Select role
Register
© Spring 2026 Real Estate Agency Portal

Login Page

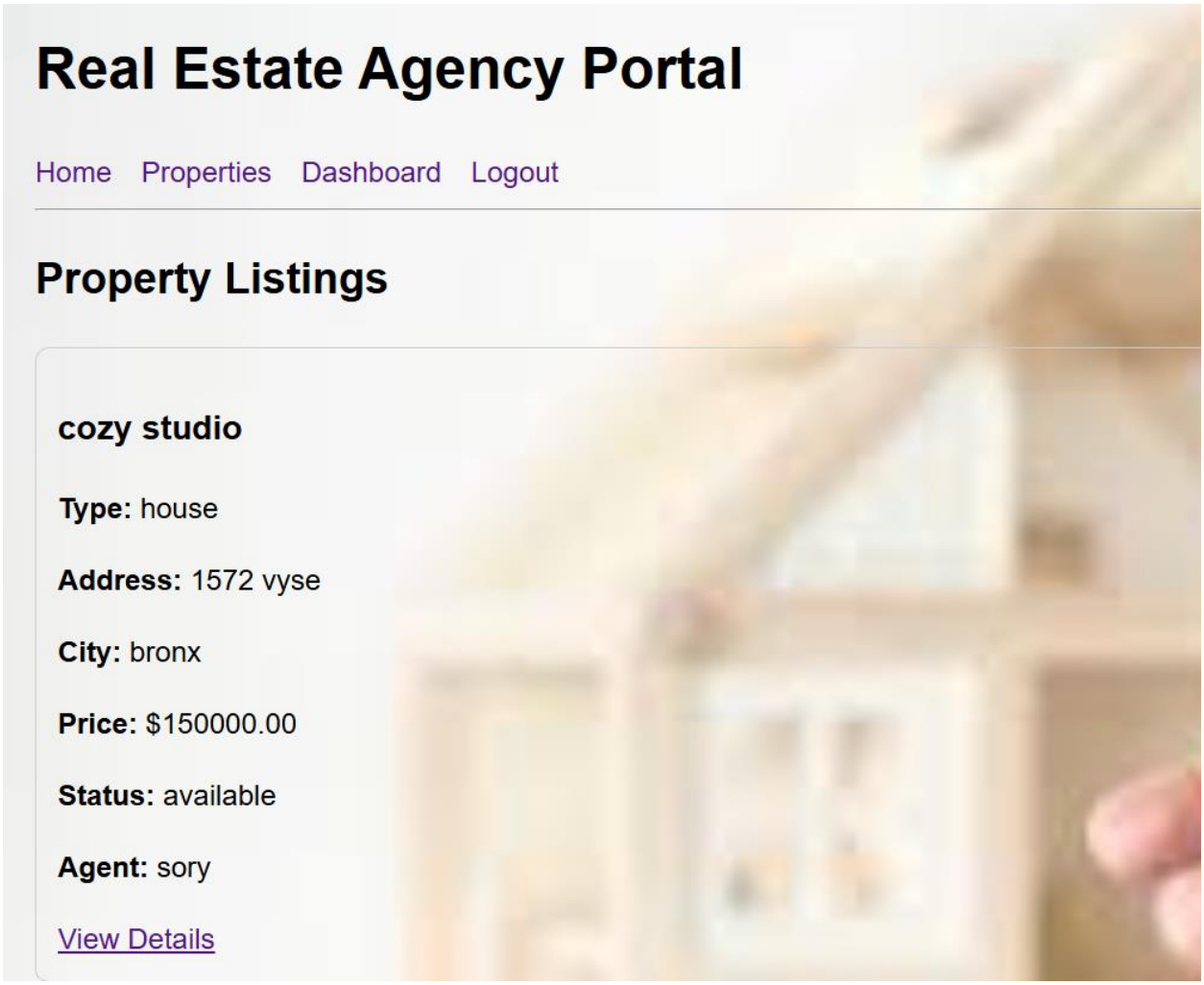

Real Estate Agency Portal
Home Properties Login Register
Login
Username
Password
Login
© Spring 2026 Real Estate Agency Portal



Add Property Page



1. **Property Listing Page**



Inquiry Submission Page

Real Estate Agency Portal

Home Properties Dashboard Logout

Dashboard

Welcome: dala

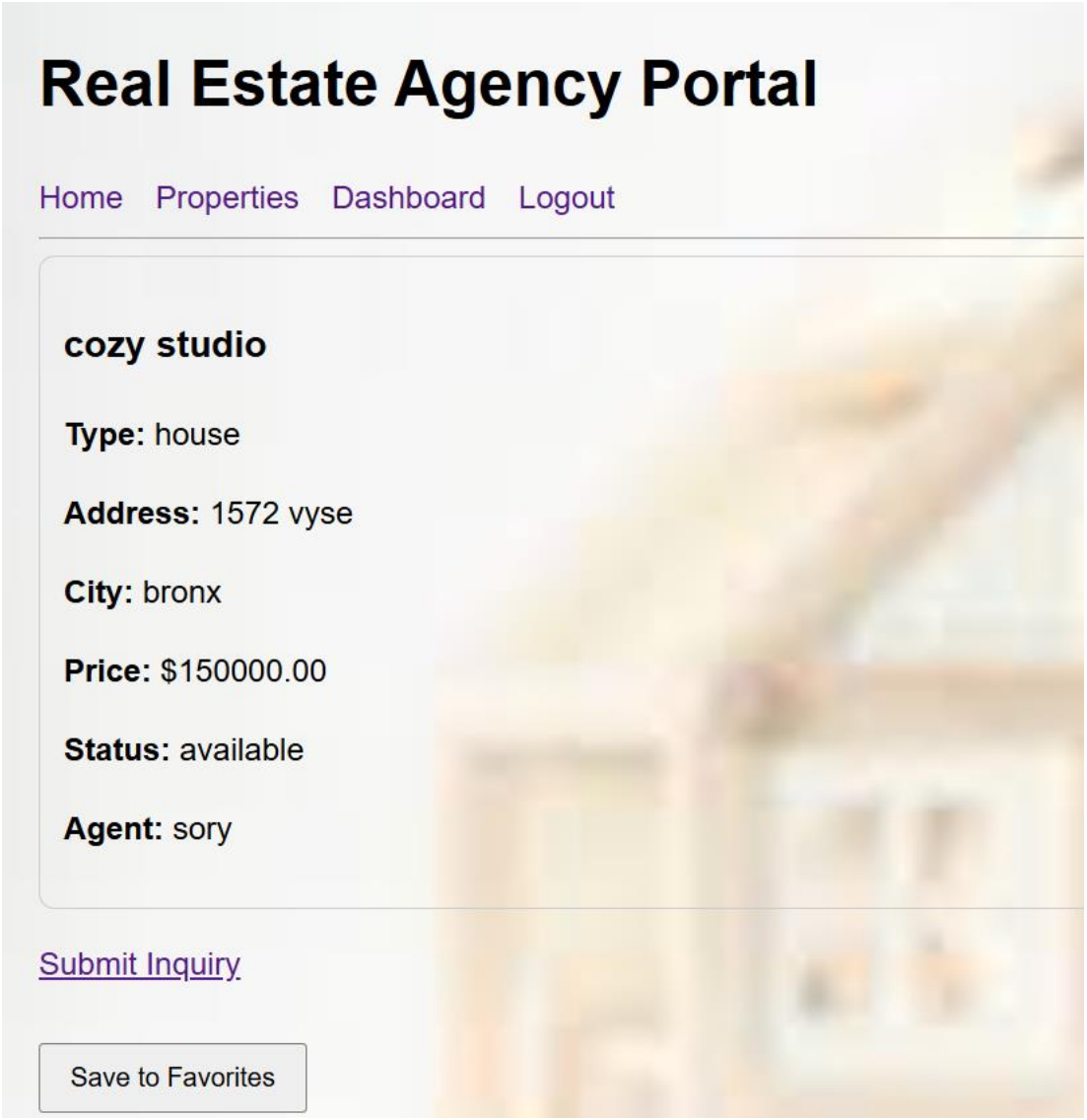
Role: renter

Common Actions

Browse Properties

Real Estate Agency Portal

Home Properties Dashboard Logout



cozy studio

Type: house

Address: 1572 vyse

City: bronx

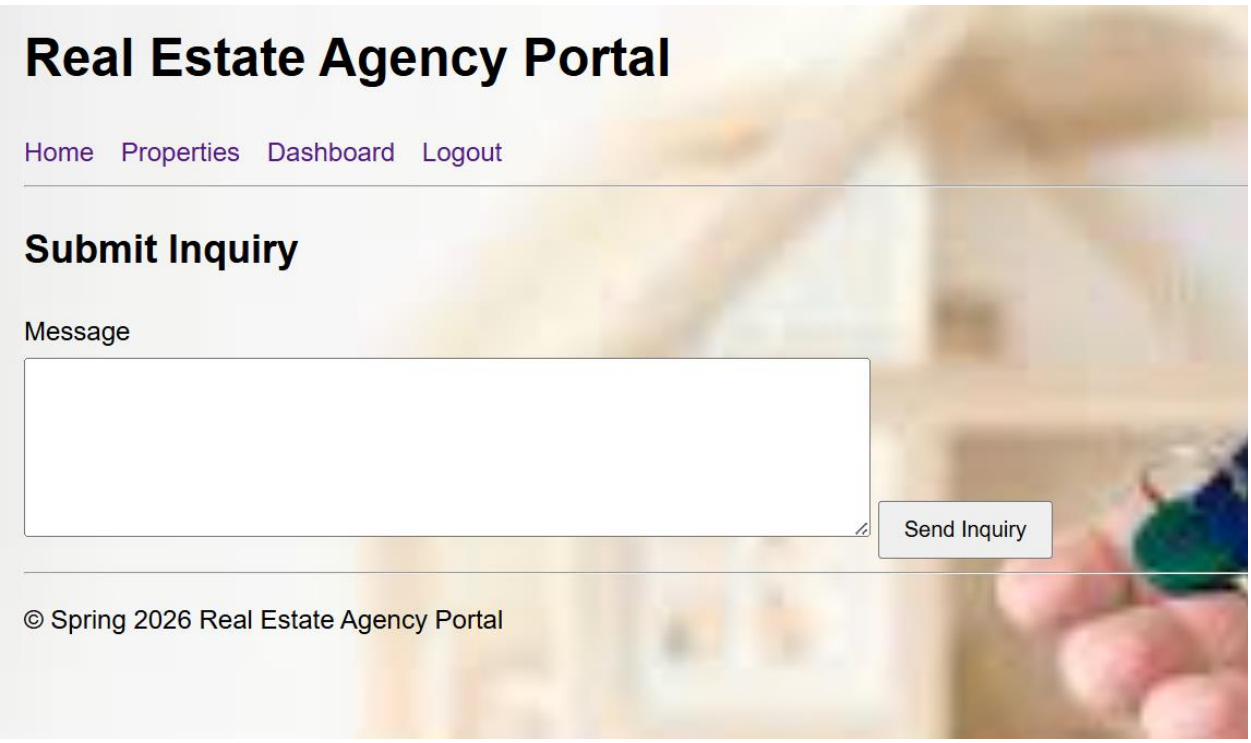
Price: $150000.00

Status: available

Agent: sory

Submit Inquiry

Save to Favorites



Database Tables (phpMyAdmin)

Users table

```sql
-- Spring 2026
-- ========================================================

CREATE DATABASE IF NOT EXISTS real_estate_portal_db;
USE real_estate_portal_db;

DROP TABLE IF EXISTS Favorites;
DROP TABLE IF EXISTS Transactions;
DROP TABLE IF EXISTS Inquiries;
DROP TABLE IF EXISTS Properties;
DROP TABLE IF EXISTS Users;

CREATE TABLE Users (
    userId INT NOT NULL AUTO_INCREMENT,
    userName VARCHAR(50) NOT NULL UNIQUE,
    contactInfo VARCHAR(200),
    passwordHash VARCHAR(255) NOT NULL,
    userType ENUM('agent', 'buyer', 'renter') NOT NULL,
    PRIMARY KEY (userId)
);
```

Properties table

```sql
CREATE TABLE Properties (
    propertyId INT NOT NULL UNIQUE AUTO_INCREMENT,
    title VARCHAR(100) NOT NULL,
    propertyType VARCHAR(50) NOT NULL,
    address VARCHAR(200) NOT NULL,
    city VARCHAR(100) NOT NULL,
    price DECIMAL(12,2) NOT NULL,
    status ENUM('available', 'sold', 'rented') NOT NULL DEFAULT 'available',
    agentId INT NOT NULL,
    Primary Key (propertyId),
    Foreign Key (agentId) references Users(userId)
);
```

Inquiries table

```sql
CREATE TABLE Inquiries (
    inquiryId INT NOT NULL UNIQUE AUTO_INCREMENT,
    userId INT NOT NULL,
    propertyId INT NOT NULL,
    message VARCHAR(255) NOT NULL,
    inquiryDate DATETIME NOT NULL,
    Primary Key (inquiryId),
    Foreign Key (userId) references Users(userId),
    Foreign Key (propertyId) references Properties(propertyId)
);
```

Transaction table

```sql
CREATE TABLE Transactions(
    transactionId INT NOT NULL UNIQUE AUTO_INCREMENT,
    propertyId INT NOT NULL,
    userId INT NOT NULL,
    transactionType ENUM('sale', 'rental') NOT NULL,
    transactionDate DATETIME NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    Primary Key (transactionId),
    Foreign Key (propertyId) references Properties(propertyId),
    Foreign Key (userId) references Users(userId)
);
```

Favorites table

```sql
CREATE TABLE Favorites(
    favoriteId INT NOT NULL UNIQUE AUTO_INCREMENT,
    userId INT NOT NULL,
    propertyId INT NOT NULL,
    savedDate DATETIME NOT NULL,
    Primary Key (favoriteId),
    Foreign Key (userId) references Users(userId),
    Foreign Key (propertyId) references Properties(propertyId)
);
```

Procedure

```
DELIMITER $$

CREATE PROCEDURE AddOrUpdateUser(
    IN uid INT,
    IN uname VARCHAR(50),
    IN contact VARCHAR(200),
    IN passHash VARCHAR(255),
    IN utype ENUM('agent','buyer','renter')
)
BEGIN
    IF uid IS NULL THEN
        INSERT INTO Users(userName, contactInfo, passwordHash, userType)
        VALUES (uname, contact, passHash, utype);
    ELSE
        UPDATE Users
        SET userName = uname,
            contactInfo = contact,
            passwordHash = passHash,
            userType = utype
        WHERE userId = uid;

CREATE PROCEDURE ProcessTransaction(
    IN propId INT,
    IN uId INT,
    IN tType ENUM('sale','rental'),
    IN amt DECIMAL(12,2)
)
BEGIN
    INSERT INTO Transactions(propertyId, userId, transactionType, transactionDate, amount)
    VALUES (propId, uId, tType, NOW(), amt);

    IF tType = 'sale' THEN
        UPDATE Properties SET status = 'sold' WHERE propertyId = propId;
    ELSE
        UPDATE Properties SET status = 'rented' WHERE propertyId = propId;
        END IF;
END $$

DELIMITER ;
CREATE VIEW PropertyListingView AS
SELECT
```

View

```
CREATE VIEW PropertyListingView AS
SELECT
    p.propertyId,
    p.title,
    p.propertyType,
    p.city,
    p.address,
    p.price,
    p.status,
    u.userName AS agentName
FROM Properties p
JOIN Users u ON p.agentId = u.userId;
DELIMITER $$
```

Trigger

```
CREATE TRIGGER AfterTransactionInsert
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    IF NEW.transactionType = 'sale' THEN
        UPDATE Properties SET status = 'sold' WHERE propertyId = NEW.propertyId;
    ELSE
        UPDATE Properties SET status = 'rented' WHERE propertyId = NEW.propertyId;
    END IF;
END $$

DELIMITER ;
```

Conclusion

This project successfully demonstrates the development of a full-stack real estate portal using PHP and MySQL. It integrates database design, backend logic, authentication, and role-based access control into a functional system. The platform reflects real-world application behavior and provides a strong foundation for further enhancements such as search features, image uploads, and admin dashboards.